Assignment 5: UML Use Case Diagram, Sudoku Puzzle Solving System

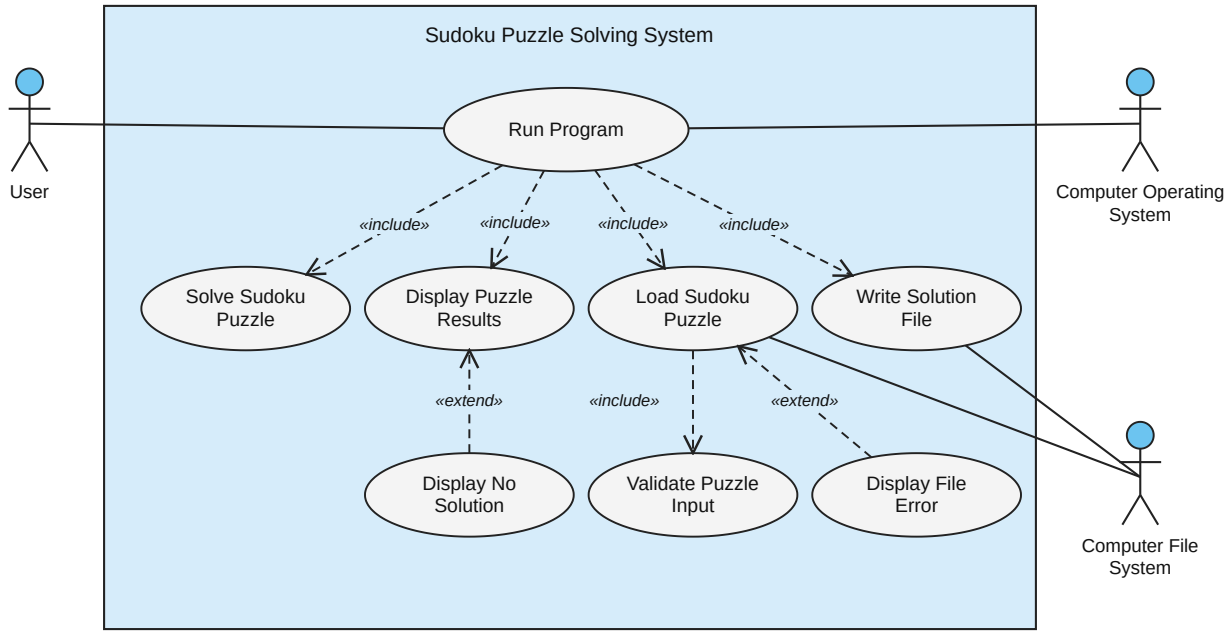
EECS 348, Software Engineering I. Revised by Andy Garcia, 10.28.26. Based on the Assignment 4 use case diagram by Ana Gonzalez Yuil.

GenAI used: Claude (Anthropic), accessed through [fill in: how you accessed it, e.g. the Claude app in a web browser].

Prompt given:

> Using the UML Use Case Diagram for a recursive, object-based C++ Sudoku solver (reads puzzle1.txt to puzzle5.txt; loads, validates, solves with depth-first search and backtracking, displays the puzzle, solutions and solution count, and writes each puzzle's results to solutionN.txt), modify it with the level of detail from the first UML Use Case Modeling lecture: a. identify/discover a use case, b. provide a brief description, c. provide a step-by-step flow (outline) of the events, d. provide a detailed description.

Use case diagram



Dashed arrows: «include» points from the base use case to the use case it includes; «extend» points from the extending use case to the base use case it extends. Solid lines connect actors to use cases.

Changes from the Assignment 4 diagram

- Added the use cases **Validate Puzzle Input** and **Write Solution File**, which cover the new input checks and the solutionN.txt output.
- Gave **Display File Error** and **Display No Solution** their own outline and detailed description (steps a to d); before, only four of the use cases had them.
- Redrew the «include» arrows so they point from the base use case to the included one, and added the File System actor to Write Solution File.
- Updated Load, Display and Run Program so they match what the Assignment 5 program does.

Use Case 1: Run Program

a. Use case identified. Run Program. **Actors:** User (primary); Computer Operating System (secondary). **Trigger:** The user wants the Sudoku puzzles solved. **Relationships:** Includes: Load Sudoku Puzzle, Solve Sudoku Puzzle, Display Puzzle Results, Write Solution File.

b. Brief description. The user starts the program, which solves the five Sudoku puzzles (puzzle1.txt to puzzle5.txt) one after another.

c. Step-by-step flow (outline)

1. Navigate to the folder containing the program and the puzzle files
2. Type the command that runs the program
3. Press Enter
4. The program processes puzzle1.txt through puzzle5.txt in order

d. Detailed description

- **Navigate:** Use the terminal's directory commands (or the file manager's "open terminal here") to reach the folder that holds Assignment5.exe and the five puzzle files.
- **Type the command:** Type the command that runs the executable, for example `./Assignment5.exe`.
- **Press Enter:** The operating system loads the executable and starts it.
- **Process the puzzles:** The program's main function loops over the five file names. For each one it runs the load, solve, display and write use cases below, then moves on to the next puzzle, even if one puzzle fails.

Use Case 2: Load Sudoku Puzzle

a. Use case identified. Load Sudoku Puzzle. **Actors:** Computer File System (secondary). **Trigger:** The program starts processing a puzzle file. **Relationships:** Includes: Validate Puzzle Input. Extended by: Display File Error.

b. Brief description. The program reads one puzzle file into a 9x9 grid so it can be checked and solved.

c. Step-by-step flow (outline)

1. Open the puzzle file
2. Read 81 cell values into the grid, row by row
3. Validate the puzzle input
4. If the input is valid, the puzzle is ready to solve

d. Detailed description

- **Open the puzzle file:** The program asks the file system to open the file (for example puzzle1.txt). If it cannot be opened, loading fails.
- **Read the values:** The program reads 81 values, one for each cell, into a list of lists (a 9x9 grid). Each value is a digit from 1 to 9 or an underscore for a blank.
- **Validate the input:** Validate Puzzle Input is run on what was read. Loading succeeds only if every check passes.
- **Ready to solve:** A valid grid is kept in the puzzle object. An invalid one triggers Display File Error.

Use Case 3: Validate Puzzle Input new in Assignment 5

a. Use case identified. Validate Puzzle Input. **Actors:** None (internal to the program). **Trigger:** Load Sudoku Puzzle has read the cell values. **Relationships:** Included by: Load Sudoku Puzzle.

b. Brief description. The program checks that a puzzle file holds one complete, legal Sudoku starting grid before any solving is attempted.

c. Step-by-step flow (outline)

1. Check that all 81 cell values could be read
2. Check that every value is 1 to 9 or an underscore
3. Check that there are no extra values after the 81st
4. Check that at least 17 cells are filled
5. Check that the given numbers do not conflict
6. Report pass or fail to Load Sudoku Puzzle

d. Detailed description

- **All 81 values:** If the file ends before 81 values are read, the puzzle is rejected.
- **Legal characters:** Any character other than 1 to 9 or an underscore (a letter, a 0) causes rejection.
- **No extra values:** After the 81st value, the program tries to read one more. If another value exists, the file holds more than a 9x9 grid and is rejected.
- **At least 17 givens:** The program counts the filled cells. Fewer than 17 means the puzzle is rejected.
- **No conflicts:** For each given number, the program checks that the same digit does not already appear in its row, column or 3x3 subgrid. Two conflicting givens mean the puzzle cannot be solved, so it is rejected.
- **Report the result:** The program returns true (valid) or false (invalid) to Load Sudoku Puzzle.

Use Case 4: Display File Error

a. Use case identified. Display File Error. **Actors:** User (sees the message); Computer File System. **Trigger:** The puzzle file cannot be opened, or Validate Puzzle Input fails. **Relationships:** Extends: Load Sudoku Puzzle.

b. Brief description. The program tells the user that a puzzle could not be used, records that no solution was found, and carries on with the next puzzle.

c. Step-by-step flow (outline)

1. Print the error message with the puzzle's file name
2. Write "No solution found." to the puzzle's solution file
3. Skip solving this puzzle
4. Continue with the next puzzle file

d. Detailed description

- **Print the error:** The program prints `Error: Could not open file <name> or puzzle input is invalid.` to the console, naming the file.
- **Write the solution file:** The program writes the single line "No solution found." to the matching solution file (for example solution3.txt for puzzle3.txt).
- **Skip solving:** No grid is shown and no search is run for this puzzle.
- **Continue:** Control returns to Run Program, which moves on to the next file. If the solution file itself cannot be created, the program prints "Could not create output file" instead and also moves on.

Use Case 5: Solve Sudoku Puzzle

a. Use case identified. Solve Sudoku Puzzle. **Actors:** None (internal to the program). **Trigger:** A puzzle was loaded and validated successfully. **Relationships:** Included by: Run Program.

b. Brief description. The program finds every solution to the puzzle using recursive depth-first search with backtracking.

c. Step-by-step flow (outline)

1. Find an empty cell
2. Try the candidate digits 1 to 9 in that cell
3. Check each candidate against its row, column and 3x3 subgrid
4. If a candidate is valid, place it and repeat for the next empty cell
5. If no empty cell remains, count a solution and record it
6. Backtrack: clear the cell and try the next candidate
7. When every possibility is tried, return the solution count

d. Detailed description

- **Find an empty cell:** The program scans the grid for the next cell holding an underscore.
- **Try candidates:** For that cell it tries 1 through 9 in order.
- **Check the rules:** A candidate is allowed only if it does not already appear in the same row, column or 3x3 subgrid.
- **Place and recurse:** A valid candidate is written into the cell, and the same steps run again on the next empty cell (the recursive call).
- **Complete grid:** If there is no empty cell, all 81 cells are filled legally, so the grid is a solution. The solution counter goes up and the grid is saved.
- **Backtrack:** After the recursive call returns, the cell is reset to empty and the next candidate is tried. This undoes dead ends and also finds every other solution.
- **Finish:** When the first cell has used all its candidates, the search ends. The count may be 0, 1 or more.

Use Case 6: Display Puzzle Results

a. Use case identified. Display Puzzle Results. **Actors:** User (reads the output). **Trigger:** Solving has finished for a puzzle. **Relationships:** Extended by: Display No Solution.

b. Brief description. The program shows the user the puzzle's file name, the unsolved puzzle, how many solutions exist, and each solution.

c. Step-by-step flow (outline)

1. Print the puzzle's file name
2. Print the unsolved puzzle
3. Print the number of solutions
4. Print each solution, numbered

d. Detailed description

- **File name:** The program prints a banner with the puzzle's file name, for example puzzle1.txt.
- **Unsolved puzzle:** The starting grid is printed, nine rows of nine cells, blanks shown as underscores.
- **Number of solutions:** The program prints "Number of Solutions:" followed by the count (0, 1 or more).
- **Print the solutions:** If the count is 1 or more, each solution grid is printed under the heading "Solution 1", "Solution 2" and so on. If the count is 0, Display No Solution runs instead.

Use Case 7: Display No Solution

a. Use case identified. Display No Solution. **Actors:** User (reads the output); Computer File System. **Trigger:** The solver finished and the solution count is 0. **Relationships:** Extends: Display Puzzle Results.

b. Brief description. When a puzzle has no solution, the program says so on the console and in the puzzle's solution file.

c. Step-by-step flow (outline)

1. Detect that the solution count is 0
2. Print "no solution found" on the console
3. Write "No solution found." as the only line of the solution file

d. Detailed description

- **Detect:** After solving, the program checks whether the solution counter is 0.
- **Console message:** The program prints "no solution found" in place of any solution grids.
- **File message:** The solution file for this puzzle contains exactly one line, "No solution found.", and nothing else.

Use Case 8: Write Solution File new in Assignment 5

a. Use case identified. Write Solution File. **Actors:** Computer File System (secondary). **Trigger:** A puzzle is being processed. **Relationships:** Included by: Run Program.

b. Brief description. The program saves the results for each puzzle in a file named solutionN.txt, where N matches the puzzle number.

c. Step-by-step flow (outline)

1. Build the solution file name from the puzzle file name
2. Open (create) the solution file
3. Write each solution as nine rows, followed by a blank line
4. Write the final line: the number of solutions found
5. Close the file

d. Detailed description

- **Build the name:** "puzzle" at the start of the file name is replaced with "solution", so puzzle3.txt becomes solution3.txt.
- **Open the file:** The program creates the solution file before loading the puzzle. If it cannot be created, an error is printed and this puzzle is skipped.
- **Write the solutions:** Each solution is written in the same layout as the puzzle files: nine rows, cells separated by spaces, with a blank line after each solution.
- **Final line:** After the last solution the program writes `Solutions found: n`, where n is the number of solutions. If there were none, the file holds only "No solution found." (see Display No Solution).
- **Close the file:** The file is closed so the contents are saved before the next puzzle starts.